

We will be looking primarily at integer arithmetic. Floating point arithmetic are typically FSMs built using integer/fixed point arithmetic.

Adders

Simplest addition operation involves two input and output bits. This operator is called a half adder.

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

Building multi-bit adders requires the ability to add an input carry.

e.g

Carry	1	1	
A	1	1	
B	0	1	
Sum	1	0	0

A full adder adds a carry in (C_{in}) along with A & B. Output carry is indicated as C_{out}

A	B	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	1	1	1

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$\text{Carry} = AB + BC_{in} + AC_{in}$$

majority gate

Mirror Symmetry

A Boolean function F is mirror symmetric if

$$F(\bar{x}) = \overline{F(x)}$$

$$\text{Sum}(A, B, C_{in}) = A \oplus B \oplus C_{in}$$

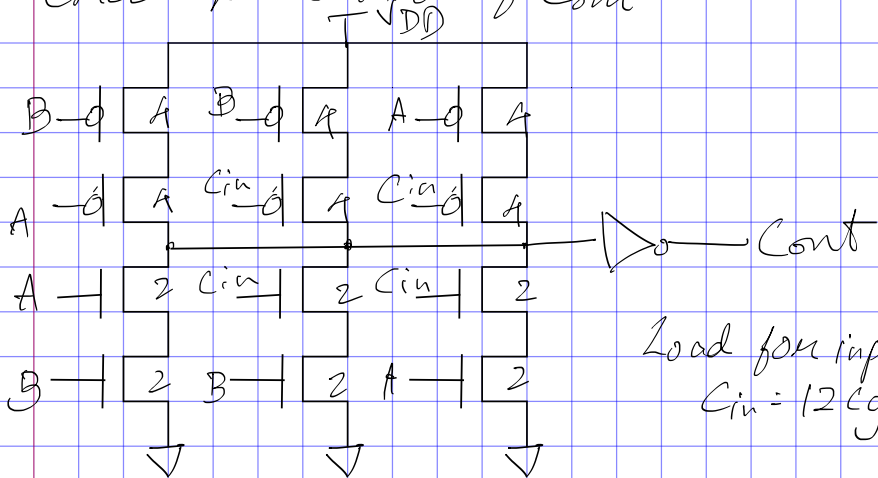
$$\text{Sum}(\bar{A}, \bar{B}, \bar{C}_{in}) = \bar{A} \oplus \bar{B} \oplus \bar{C}_{in} = \overline{A \oplus B \oplus C_{in}} = \overline{\text{Sum}}$$

$$\begin{aligned} \text{Cont}(A, B, C_{in}) &= AB + BC_{in} + AC_{in} \\ &= ABC_{in} + A\bar{B}\bar{C}_{in} + \bar{A}BC_{in} + \bar{A}\bar{B}\bar{C}_{in} \end{aligned}$$

$$\begin{aligned} \text{Cont}(\bar{A}, \bar{B}, \bar{C}_{in}) &= \bar{A}\bar{B}\bar{C}_{in} + \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + \bar{A}B\bar{C}_{in} \\ &= \overline{\text{Cont}(A, B, C_{in})} \end{aligned}$$

PUN & PDN are duals in the CMOS implementation.
Mirror symmetry property implies a function is its dual.

CMOS implementation of Cont.



HW Do the CMOS implementation of SUM.

Carry evaluation is critical in the adder. Addition of the i th bit requires the carry from $(i-1)$ th bit to be ready.

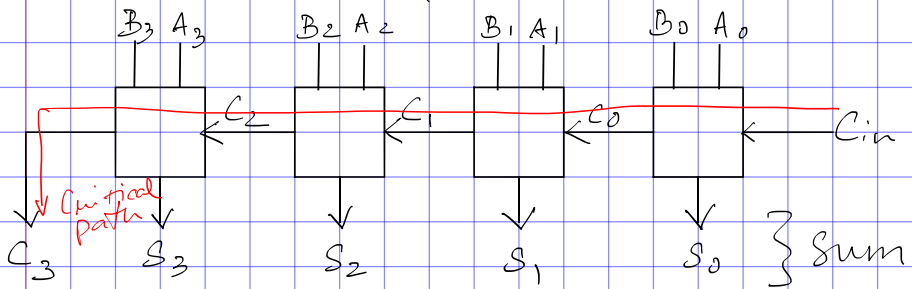
e.g. C_{in} transitions from 0 $\rightarrow$ 1

A	0	0	0	0	1	0	0	1	1	1
B	0	0	0	0	0	0	0	0	0	0
C_{in}	0					1				
Sum	0	1	1	1	1	1	0	0	0	0

Carry has to "ripple" through to evaluate the sum. $\Rightarrow$ Carry propagation forms the critical path in a simple "ripple carry adder".

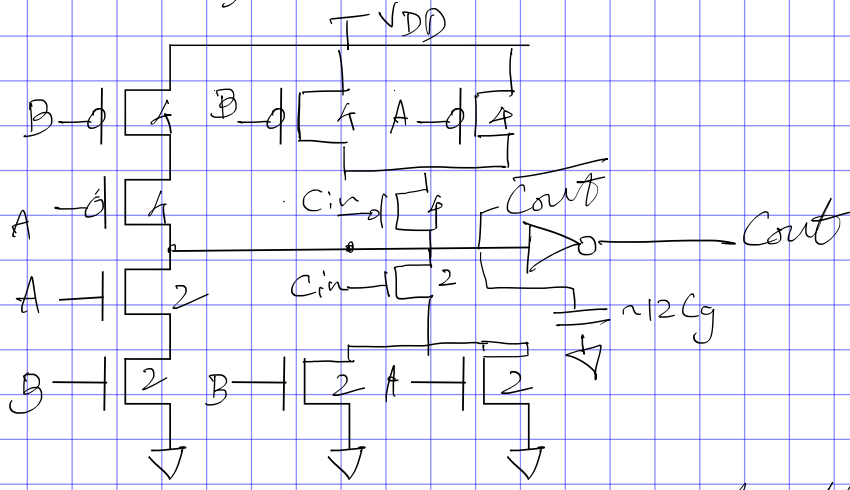
Ripple carry adder

4-bit adder.



Since the carry evaluation determines the speed of the adder, it makes sense to ensure carry logic is as fast as possible. To do this we will look at multiple optimisations ranging from the full adder all the way to the complete adder.

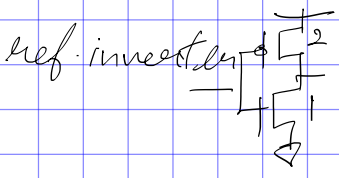
1.) Refactoring (shared logic)
 We first modify the carry implementation so that the load seen by C_{in} is reduced.



C_{in} sees a load of $6C_g$ in the above implementation in place of the $12C_g$ seen earlier.

$$C_{out} = C_{in}(A+B) + AB$$

logical effort and parasitic delay of C_{in}



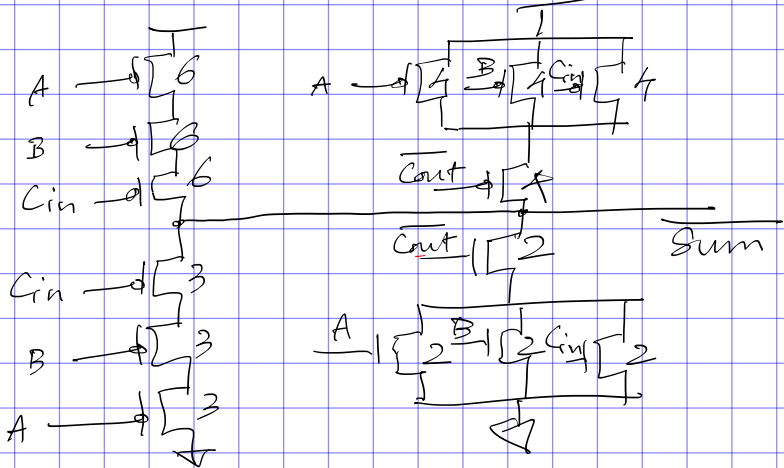
$$g_{Cin} = \frac{6C_g}{3C_g} = 2$$

$$p_{Cin} = \frac{12C_g}{3C_g} = 4$$

Since Sum is not in the critical path, it can be rewritten as

$$\begin{aligned} Sum &= A \oplus B \oplus C_{in} \\ &= ABC_{in} + \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} \\ &= ABC_{in} + (\bar{A}\bar{B} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in}) \\ &\quad \quad \quad \downarrow \text{Cout (Symmetry)} \\ &= ABC_{in} + \bar{C}_{out}(A+B+C_{in}) \end{aligned}$$

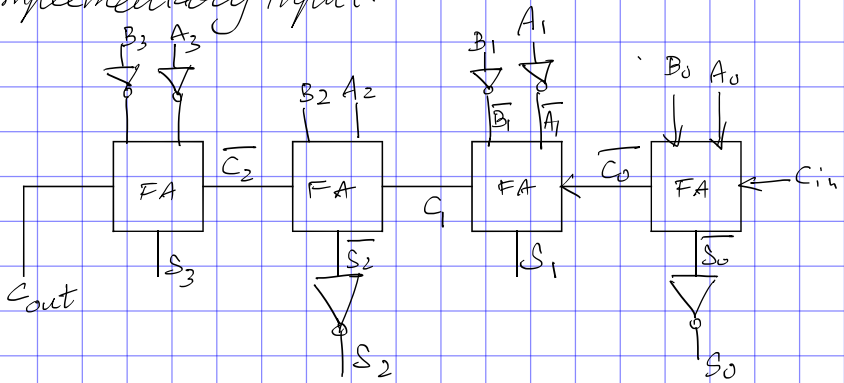
$$Sum = \underbrace{ABC_{in}}_n + \bar{C}_{out}(A+B+C_{in})$$



2.) Microsymmetry

All bits of A & B are typically available at the same instant and the delay of carry block is greater than that of an unit inverter.

Using this fact along with the mirror symmetry property of the carry/sum logic implies alternate full adders can use complementary input.



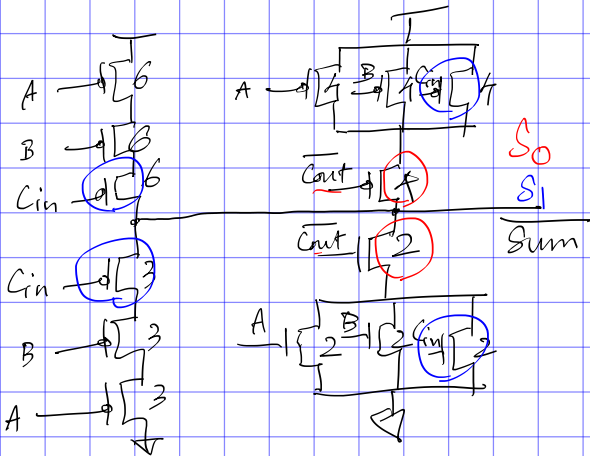
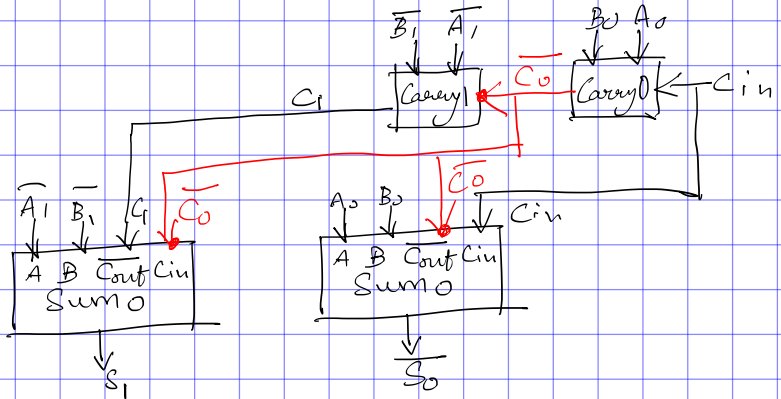
Let t_{carry} and t_{sum} represent the time required by the full adder to generate carry and sum respectively.

t_{adder} = critical path delay of the adder

For an N-bit adder, $t_{adder} = (N-1)t_{carry} + t_{sum}$
 $+ \begin{cases} t_{inv} & \text{if } N \text{ is odd} \\ 0 & \text{if } N \text{ is even.} \end{cases}$

3.) Sizing the SUM block.

In the ripple carry each carry has a fan-out of three as shown below in *red*.



The total output capacitance of carry

$$= (6 + 3 + 4 + 2) \text{ } \left. \vphantom{\begin{matrix} 6 \\ 3 \\ 4 \\ 2 \end{matrix}} \right\} \text{ from } S_0$$

$$+ (4 + 2) \text{ } \left. \vphantom{\begin{matrix} 4 \\ 2 \end{matrix}} \right\} \text{ from } S_1$$

$$+ (4 + 2) \text{ } \left. \vphantom{\begin{matrix} 4 \\ 2 \end{matrix}} \right\} \text{ from } C_1$$

We want to size the carry chain to minimise delay. Let 's' be the size of all carry blocks.

$$\Rightarrow \text{Output load} = 21 + 6s = C_{\text{load}}$$

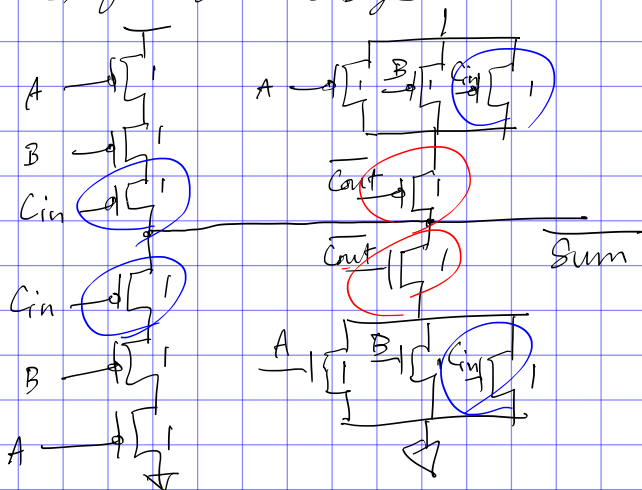
$$\text{Electrical effort of carry} = \frac{21 + 6s}{6s}$$

$$g_{\text{carry}} = 2$$

$$\text{Optimal stage effort} = 4 = g \times h = 2 \times \frac{21 + 6s}{6s}$$

$$\boxed{s = 3.5}$$

Instead of sizing the sum block to match ref. inverter resistance, if we were to size it as below: $C_{\text{load}} = 6 + 6s$



For such a case: $g \times h = 4$

$$2 \times \frac{6 + 6s}{6s} = 4$$

$$\boxed{s = 1}$$

We forego the speed of sum to achieve smaller carry block which reduces overall area/wirelength.

An alternate way to look at the adder is in terms of carry generation, and propagation.

If $A=B=1$ in a full adder $C_{out}=1$ irrespective of C_{in} . Carry is generated in this case.

If $A=1, B=0$ or $A=0, B=1$, $C_{out} = C_{in}$. Carry is propagated in this case.

Generate $G_i = A \cdot B$

Propagate $P_i = A \oplus B$.

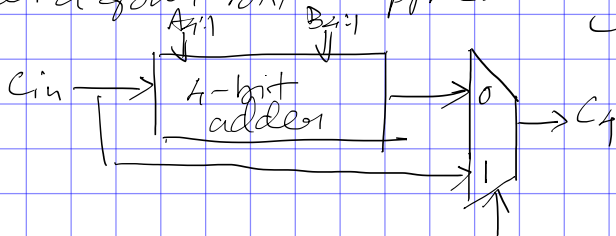
C_{out} can be rewritten as:

$$C_{out} = A \cdot B + C_{in}(A \oplus B)$$

$$= G_i + C_{in} \cdot P_i$$

$$Sum = P_i \oplus C_{in}$$

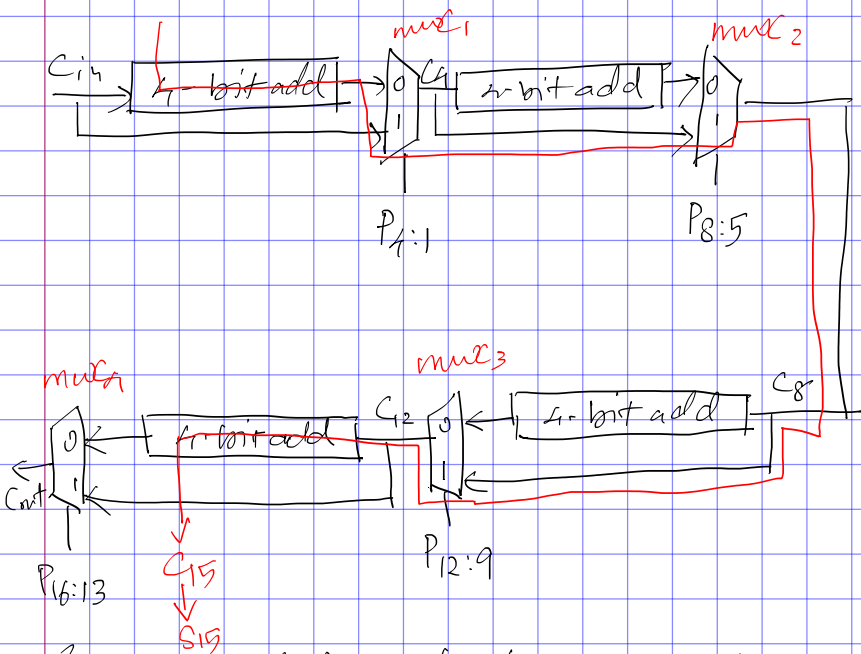
Consider a four-bit ripple-carry adder:



$$P_{4:1} = P_4 \cdot P_3 \cdot P_2 \cdot P_1$$

If all the 4-bits are in propagate mode then $C_4 = C_{in}$. The multiplexer helps evaluate C_4 quickly in such a case.

16-bit carry bypass adder.



Let t_{carry} & t_{sum} be the carry & sum delay. of full adder & t_{mux} be the delay of the mux.

the worst case time to generate C_{out} is carry being generated by (A_1, B_1) and propagated to (A_4, B_4) and then propagate through mux1, followed by bypass through mux2, mux3, and mux4.

$$t_{Cout} = 4 \times t_{carry} + 4 t_{mux}.$$

C_{out} calculation is no longer the critical path in the bypass adder.

S_{16} evaluation requires C_{15} . C_{15} evaluation takes the path shown in red in the figure.

$$t_{C_{15}} = 4 t_{carry} + 3 t_{mux} + 3 t_{carry}.$$

$$t_{S_{15}} = \underbrace{\hspace{10em}}_{\downarrow} + t_{sum}$$

S_{15} evaluation forms the critical path.

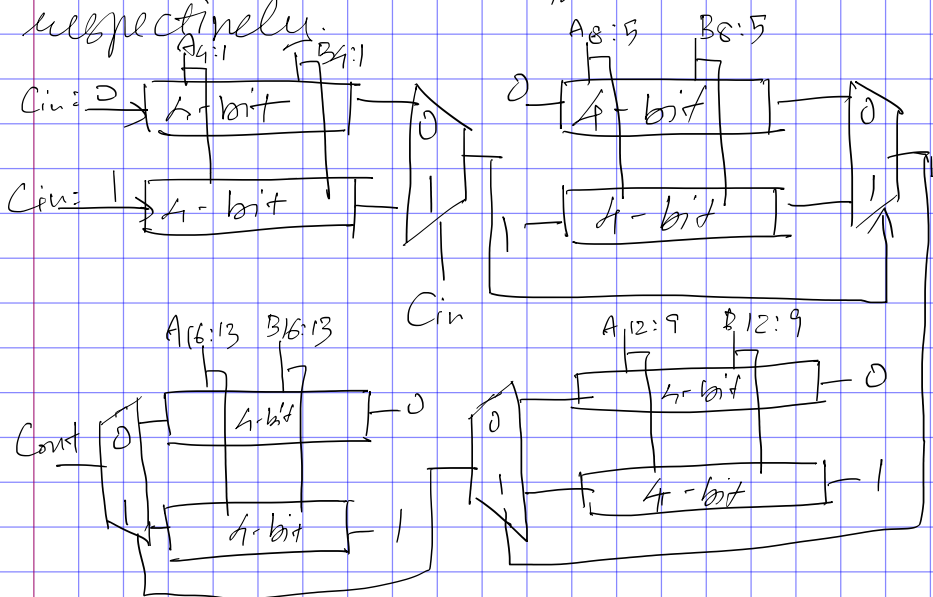
For an N -bit carry bypass adder which has ' M ' bit ripple carry adder in each stage the critical path delay is

$$t_{\text{carry bypass}} = M \times t_{\text{carry}} + \left(\frac{N}{M} - 1 \right) t_{\text{mux}} + (M-1) t_{\text{carry}} + t_{\text{sum}}$$

Delay of this adder is still a linear function of the number of bits. Can we build adders that are sub-linear?

Carry select adder

Consider two copies of ripple carry adders that evaluate with $C_{in} = 0$ and $C_{in} = 1$ respectively.



All the 4-bit adders evaluate in parallel and are ready with their output carry in $4 \times t_{\text{carry}}$ time. Once the previous stages carry selects the appropriate output in the muxes, that stage's output is ready.
 $\Rightarrow t_{\text{carry select}} = 4 \times t_{\text{carry}} + 4 \times t_{\text{mux}}$

Square root adder

In the carry select adder, second stage is waiting for t_{mux} time after it has completed for the previous stages carry to come through and choose the right output. Similarly the third stage is waiting for $2 \times t_{mux}$ time and so on.

If $t_{mux} \approx t_{carry}$ then the second stage could have a 5-bit adder instead of waiting and the third stage 6-bits and so on.

For an N -bit adder with P stages

$$N = 4 + 5 + 6 + 7 \dots 4 + (P-1)$$

$$N = 4 + \frac{P(P-1)}{2}$$

$$\text{if } N \gg 4, \text{ then } P \approx \sqrt{2N}$$

$$t_{sq. adder} = 4t_{carry} + P \times t_{mux}.$$

$$\text{Ripple carry adder delay} = f(N) = (N-1)t_{carry} + t_{sum}$$

$$\text{Carry bypass } t_d = f(N) = (2M-1)t_{carry} + \left(\frac{N-1}{M}\right)t_{mux} + t_{sum}$$

$$\text{Square root adder} = 4t_{carry} + P t_{mux}$$

where $N = 4 + \frac{P(P-1)}{2}$

Can we do better than square root?

Carry lookahead adders

Propagate & generate signals

$$P_i = A_i \oplus B_i, \quad G_i = A_i \cdot B_i$$

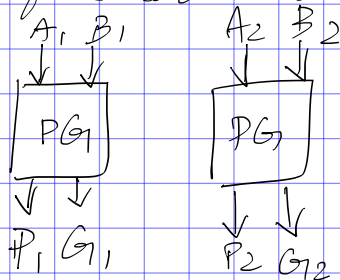
$$C_i = G_i + P_i C_{in} \quad \text{carry out}$$

$$S_i = P_i \oplus C_{in} \quad \text{sum out}$$

For each bit in an adder

$P_i = A_i \oplus B_i$ & $G_i = A_i \cdot B_i$ can be simultaneously evaluated.

Consider two full adders treated as a block.



Let C_{in} be the input carry to the 2-bit adder.

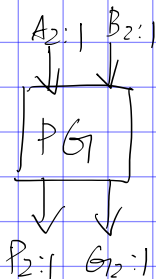
$$C_1 = G_1 + P_1 \cdot C_{in}$$

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 C_{in}$$

$$C_2 = \underbrace{(G_2 + P_2 G_1)}_{\text{Block generate}} + \underbrace{(P_2 P_1)}_{\text{Block propagate}} C_1$$

Block generate Block propagate

The two signals can be treated as block generate and propagate.



$$P_{2:1} = P_2 \cdot P_1$$

$$G_{2:1} = G_2 + P_2 G_1$$

Similarly $G_{3:1} = G_3 + P_3 G_2 + P_3 P_2 G_1$

$$P_{3:1} = P_3 \cdot P_2 \cdot P_1$$

Dot operator (•)

$$\begin{aligned} (G_{2:1}, P_{2:1}) &= (G_2, P_2) \bullet (G_1, P_1) \\ &= (G_2 + P_2 G_1, P_2 \cdot P_1) \end{aligned}$$

$$\begin{aligned} (G_3, P_3) \bullet (G_{2:1}, P_{2:1}) &= (G_{3:1}, P_{3:1}) \\ &= (G_3 + P_3 G_2 + P_3 P_2 G_1, \\ &\quad P_3 P_2 P_1) \end{aligned}$$

Cont of a block is written as:
 $\text{Cont} = G_{12:1} + P_{2:1} \cdot C_{in}$

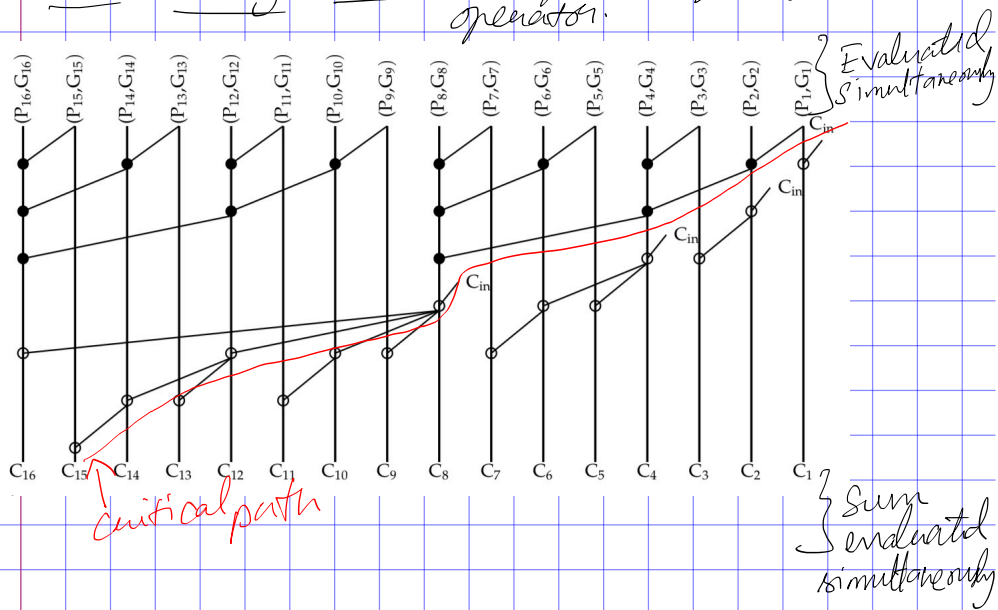
Cont generation this way is written as a circle $\odot$ operator.

$$(G_{12:1}, P_{2:1}) \odot (C_{in}, 0) = (G_{12:1} + P_{2:1} \cdot C_{in}, 0)$$

$\odot$ operator $\equiv$ $\bullet$ operator with $(C_{in}, 0)$

$$\left. \begin{aligned} (G_{i:k}, P_{i:k}) \bullet (G_{(k-1):l}, P_{(k-1):l}) \\ = (G_{i:l}, P_{i:l}) \end{aligned} \right\} \begin{array}{l} \text{Combining} \\ \text{block signals} \end{array}$$

Brent-Kung adder: max fan-out of 2 for any operator.



C_g has 4 $\odot$ and 3 $\bullet$ operators.

$$t_{BK} = t_{PG} + 4 \times t_{\odot} + 3 \times t_{\bullet} + t_{sum}$$

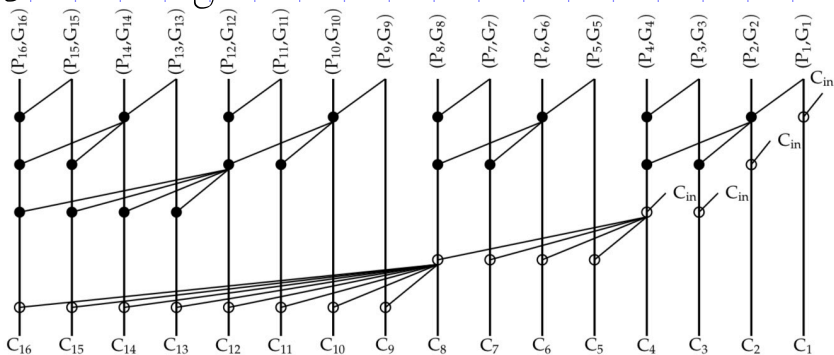
$$t_{\odot} = t_{\bullet} \Rightarrow t_{BK} = t_{PG} + 7t_{\odot} + t_{sum}$$

For an N -bit adder,

$$t_{BX} = t_{PG1} + (2 \times \log_2 N - 1) t_{\bullet} + t_{sum}$$

Multiple possible tree implementations depending on the fan-out limit.

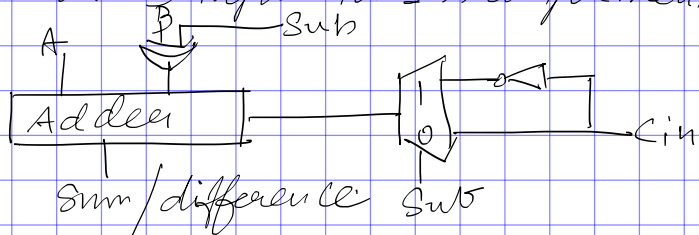
eg. Sklansky adder has exponentially growing fanout.



and Kogge-Stone has linear $\bullet$ operators in each stage and reduces the log factor

$$t_{Kogge} \sim t_{PG1} + (\log_2 N) t_{\bullet} + t_{sum}$$

Subtractor is implemented using the adder with the B input in 2's complement form.

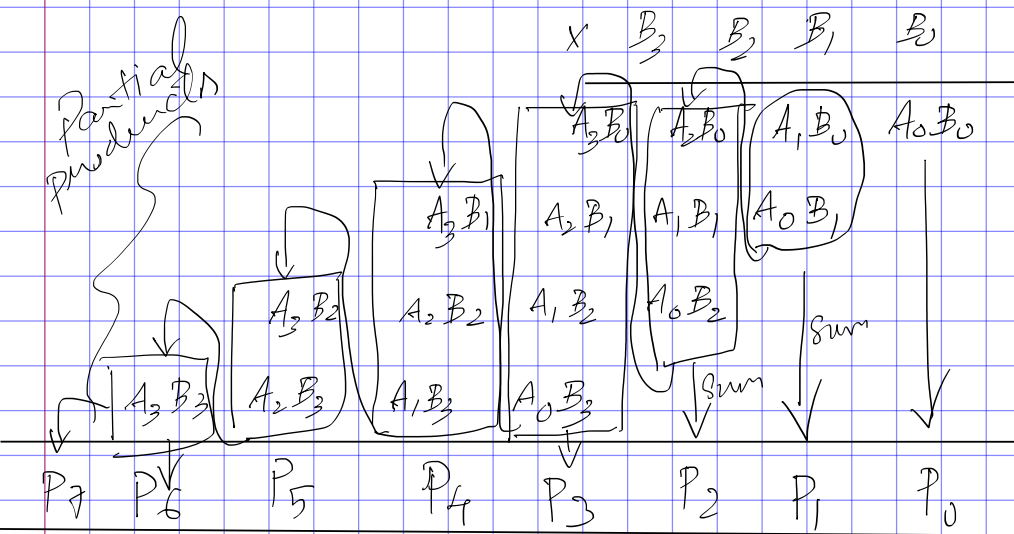


Multiplexers

Let us first look at multiplication of positive integers.

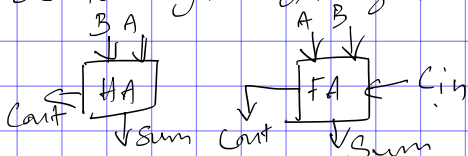
Consider two 4-bit numbers $A_{3:0}, B_{3:0}$

$A_3 \ A_2 \ A_1 \ A_0$

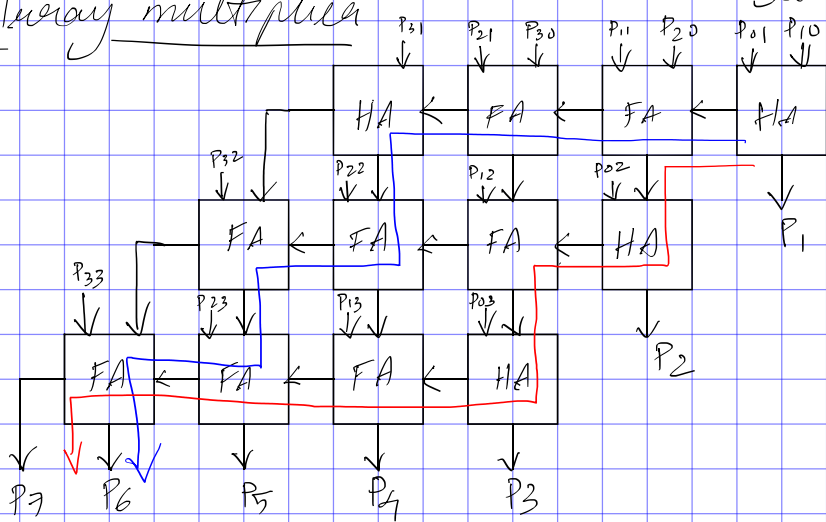


Addition of partial products using half and full adders. Let P_{ij} be the j th bit of i th partial product.

$$P_{ij} = A_i \cdot B_j$$



Array multiplier



All the partial product bits are evaluated simultaneously by ANDing the respective bits of inputs.

Array multiplier shown above has multiple critical paths that contain 5 carry and 3 sum.

$$t_{mult} = 5t_{carry} + 3t_{sum} + t_{AND}$$

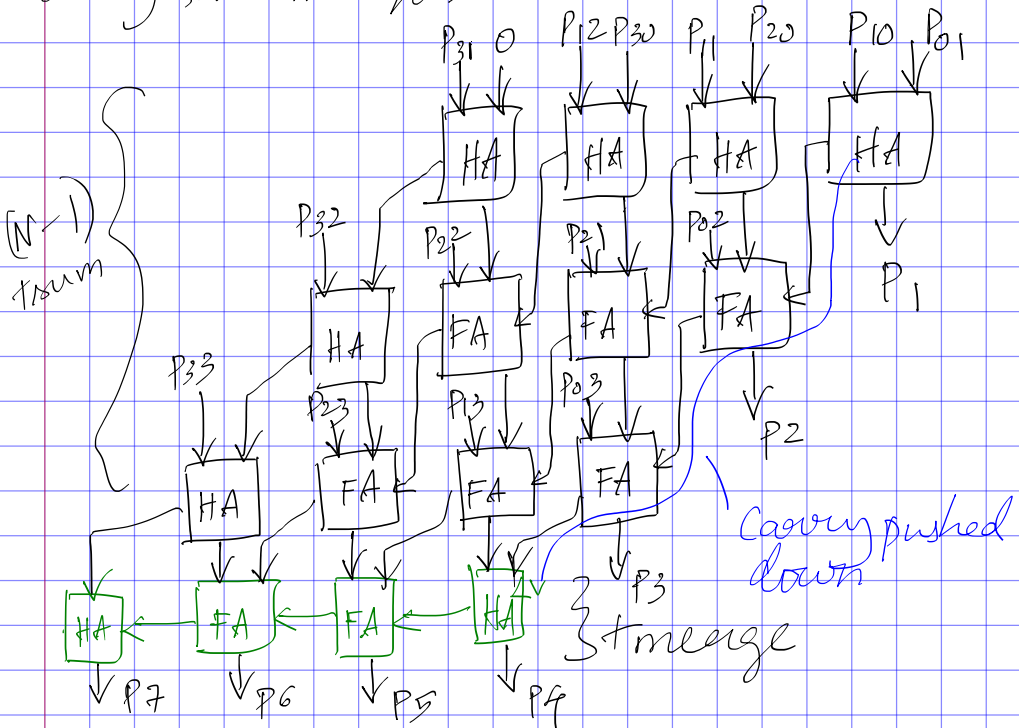
(All left going signals are carry and down going ones are sum).

An N -bit multiplier has

$$t_{mult} = t_{AND} + (2N-3)t_{carry} + (N-1)t_{sum}$$

Carry same multiplier:

Instead of pushing the carry "left" in the array multiplier, it is pushed "down" in the carry same multiplier.



This creates "1" extra stage compared to array multiplier. But the inputs to the last stage shown in green all arrive almost in $(N-1) t_{sum}$ time. The last stage can hence be thought of as a stand alone adder. The tree adders constructed earlier can be used to perform this "merge" addition operation.

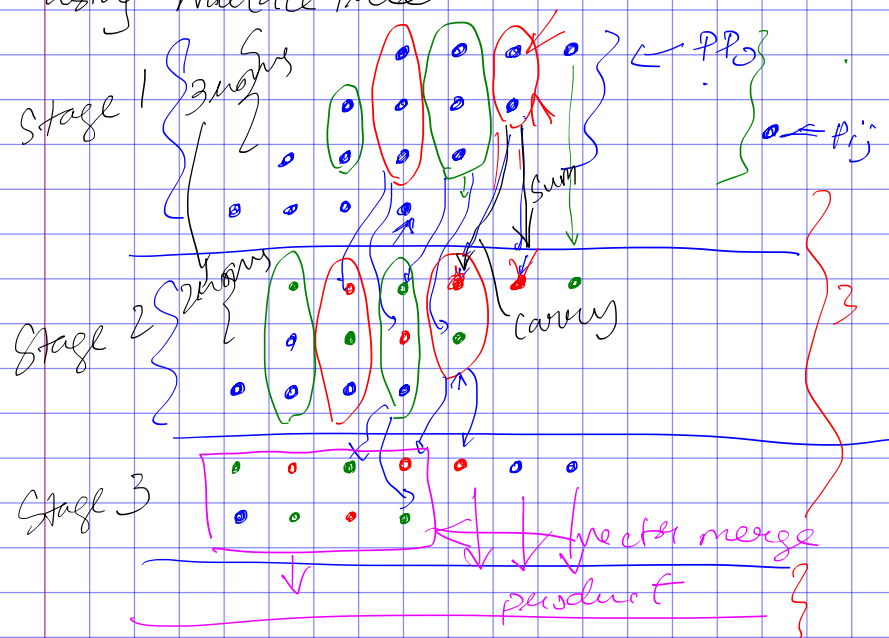
$$t_{carrysame} = t_{AND} + (N-1) t_{sum} + t_{merge}$$

$\downarrow$
 $f(\log_2 N)$

Is a logarithmic approach similar to that of a tree adder possible for the multiplier? It turns out to be possible by treating the full adder as a "compressor". A full adder takes in three input bits and "compresses" it to two output bits. Tree multipliers extend this "compression" and achieve a logarithmic number of stages.

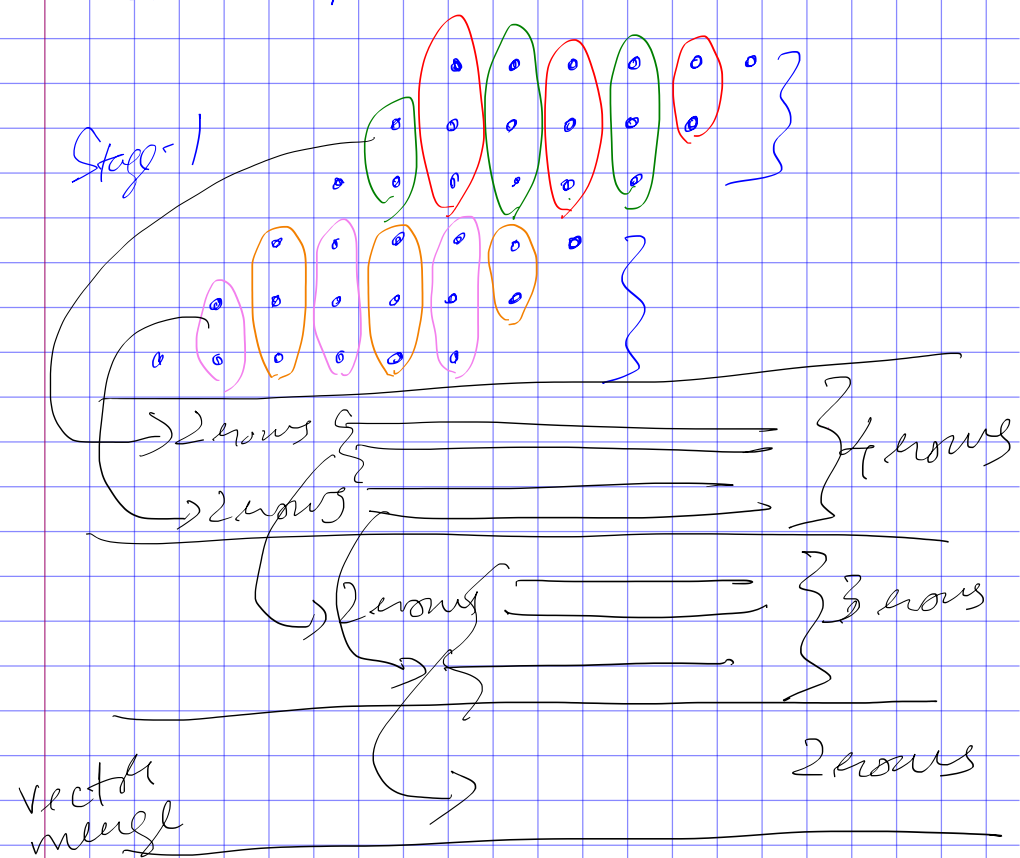
Wallace tree

In this tree structure, three partial products are merged into two recursively. Below is an illustration of a 4×4 multiplier using Wallace tree.



Each $\bullet$ represents a partial product bit p_{ij} . The colored ovals represent half/full adders.

6×6 multiplier - Wallace tree



Let h_i represent the number of rows in i^{th} stage.

$$h_{i+1} = 2 \times \underbrace{\left\lfloor \frac{h_i}{3} \right\rfloor}_{\text{floor}} + \underbrace{(h_i \% 3)}_{\text{remainder}}$$

$$\Rightarrow \text{number of stages} \approx \log_{\frac{3}{2}} N$$

$$\Rightarrow t_{\text{mult}} = t_{\text{AND}} + f(\log_{\frac{3}{2}} N) + t_{\text{merge}}$$

Dad's Tree

Another tree implementation that reduces the number of stages in the ratio of $(\frac{3}{2})$.

$$h_{i+1} = \left\lfloor \frac{h_i \times 3}{2} \right\rfloor$$

e.g. 9 rows

↓
6 rows

↓
4 rows

↓
3 rows

↓
2 rows } ready to merge

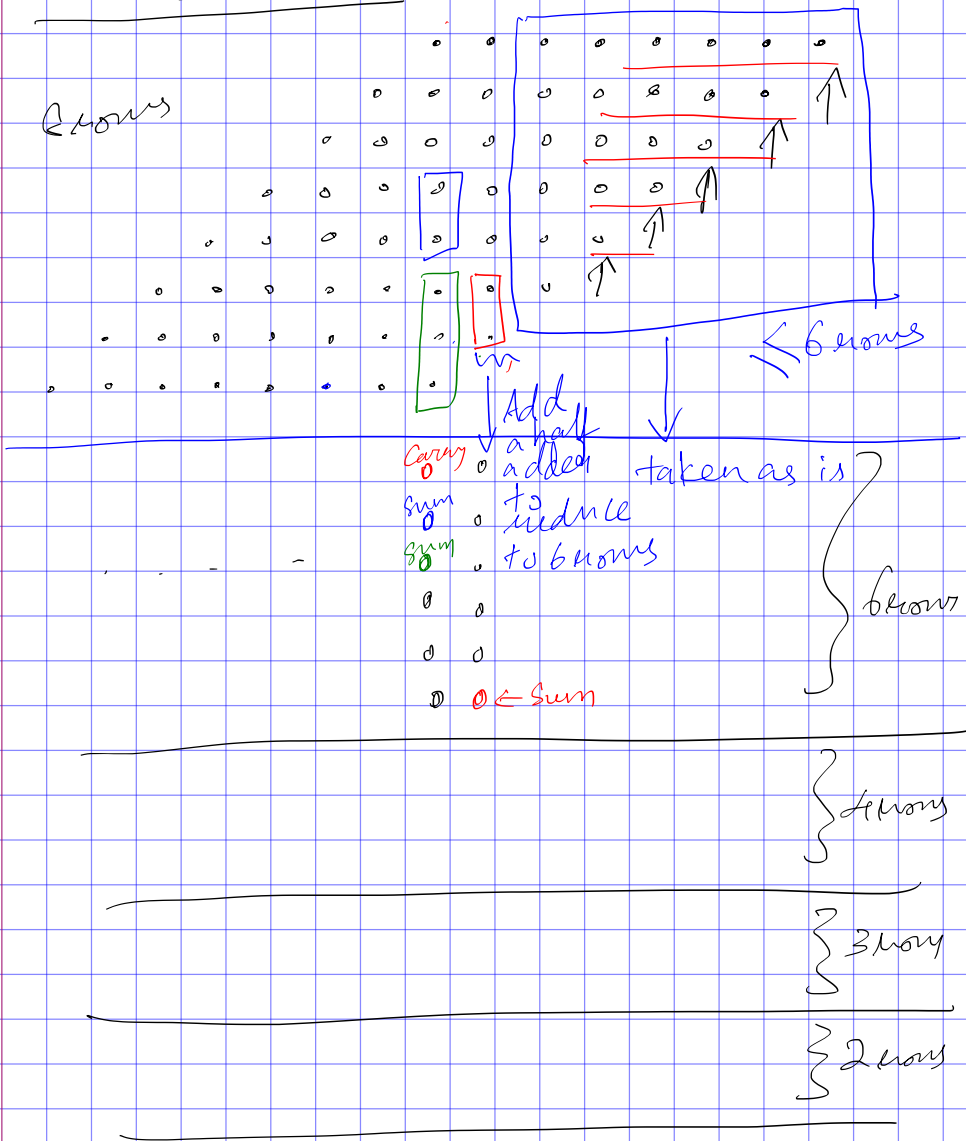
If an N lies between h_i & h_{i+1} then the number of rows to reduce is chosen as h_i in the first stage.
For e.g. 8-bit multiplier.
 $6 \leq 8 \leq 9 \Rightarrow$

$8 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$

$$\text{Number of stages} \approx \log_{\frac{3}{2}} N$$

8-bit Dadda tree

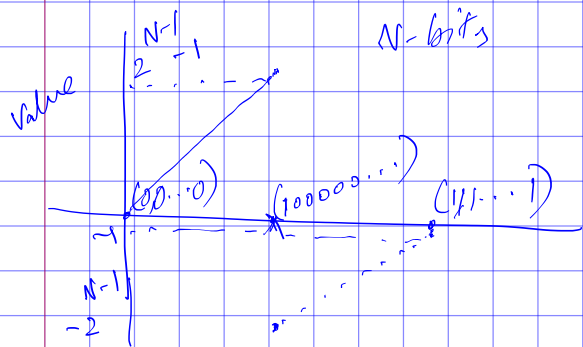
Rows



Reduce each column to the required number of rows by adding appropriate number of full and half adders considering the carry from the adjacent column.

Signed multiplication

2's complement to represent negative numbers.



$$\begin{aligned} & \downarrow \\ & 011 - 3 \\ & 100 - (-4) \\ & \left[\begin{aligned} & -2 \times b_{N-1} \\ & + \sum_{i=0}^{N-2} 2^i b_i \end{aligned} \right] \end{aligned}$$

$$a = \left(\sum_{i=0}^{N-2} a_i \times 2^i - 2^{N-1} a_{N-1} \right)$$

$$b = \left(\sum_{i=0}^{N-2} b_i \times 2^i - 2^{N-1} b_{N-1} \right)$$

} 2's complement representation

$$a \times b = \sum_{k=0}^{N-2} \sum_{i=0}^k a_i b_k 2^{i+k}$$

$$\begin{aligned} & - \sum_{i=0}^{N-2} a_{N-1} b_i 2^{N+i-1} \\ & - \sum_{i=0}^{N-2} b_{N-1} a_i 2^{N+i-1} \end{aligned}$$

$$+ a_{N-1} b_{N-1} 2^{2N-2}$$

4-bit multiplication

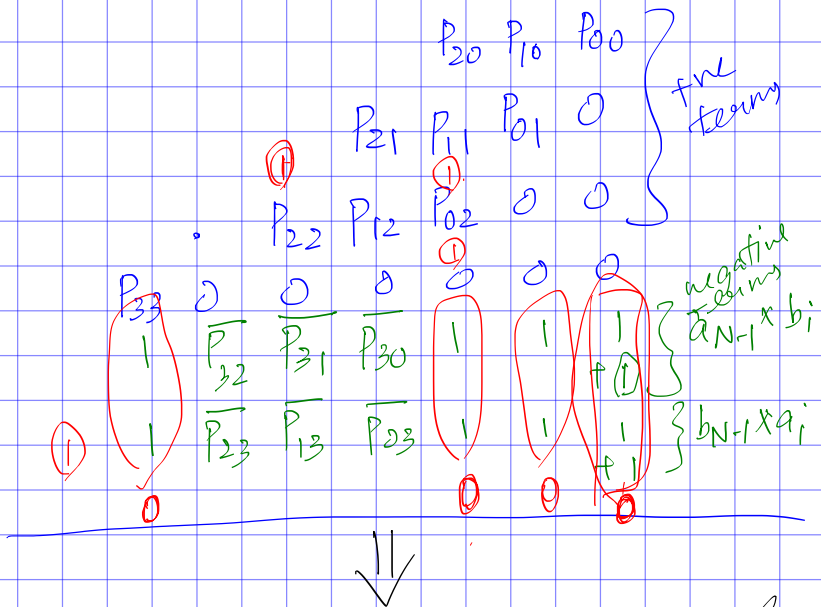
$$\begin{array}{cccc} a_3 & a_2 & a_1 & a_0 \\ b_3 & b_2 & b_1 & b_0 \end{array}$$

$$p_{ij} = a_i b_j$$

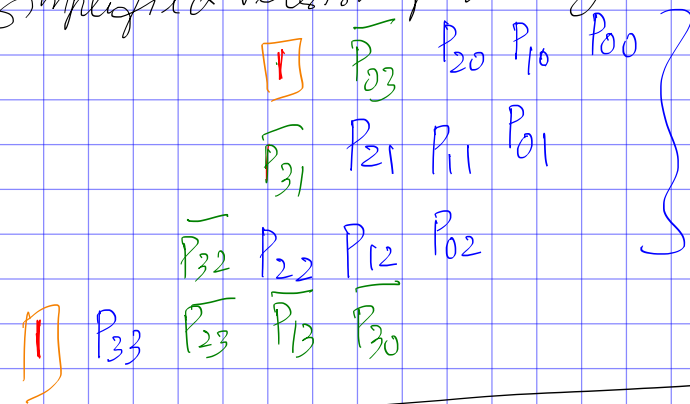
$$\begin{array}{cccccc} & & p_{20} & p_{10} & p_{00} & \\ & p_{21} & p_{11} & p_{01} & 0 & \\ & p_{22} & p_{12} & p_{02} & 0 & 0 \\ p_{33} & 0 & 0 & 0 & 0 & 0 \end{array}$$

$$\begin{array}{cccccc} 1 & \overline{p_{32}} & \overline{p_{21}} & \overline{p_{30}} & 1 & 1 \\ 1 & \overline{p_{23}} & \overline{p_{13}} & \overline{p_{03}} & 1 & 1 \end{array} \left. \begin{array}{l} \{ a_{N-1} \times b_i \} \\ \{ b_{N-1} \times a_i \} \end{array} \right\} \begin{array}{l} 2's \\ complement \end{array}$$

Precompute the constants and add the carry to the appropriate bit positions



Simplified version of the signed multiplier



HW Implement a 6x6 signed multiplier.